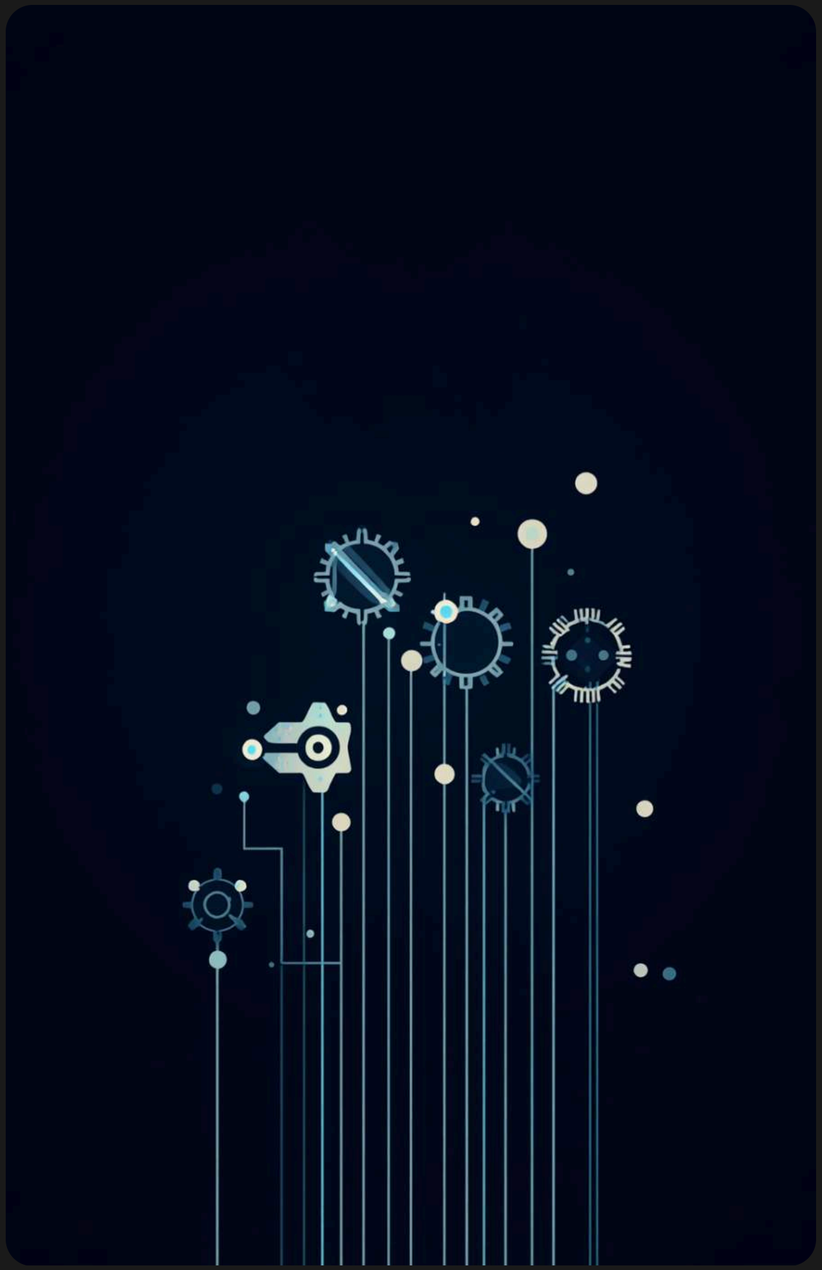
Deadlock Simulator

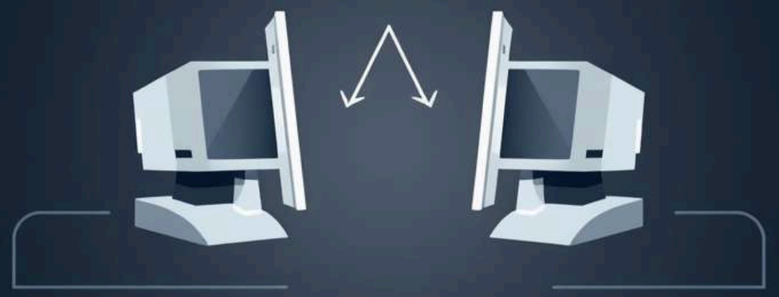
Understanding & Visualizing Deadlocks in Operating Systems

Presented by
P.Donica - 2420030655
G.Hasini-2420030735
M.Prashamsa - 2420030692
V.Sriram - 2420090126



What is a Deadlock?

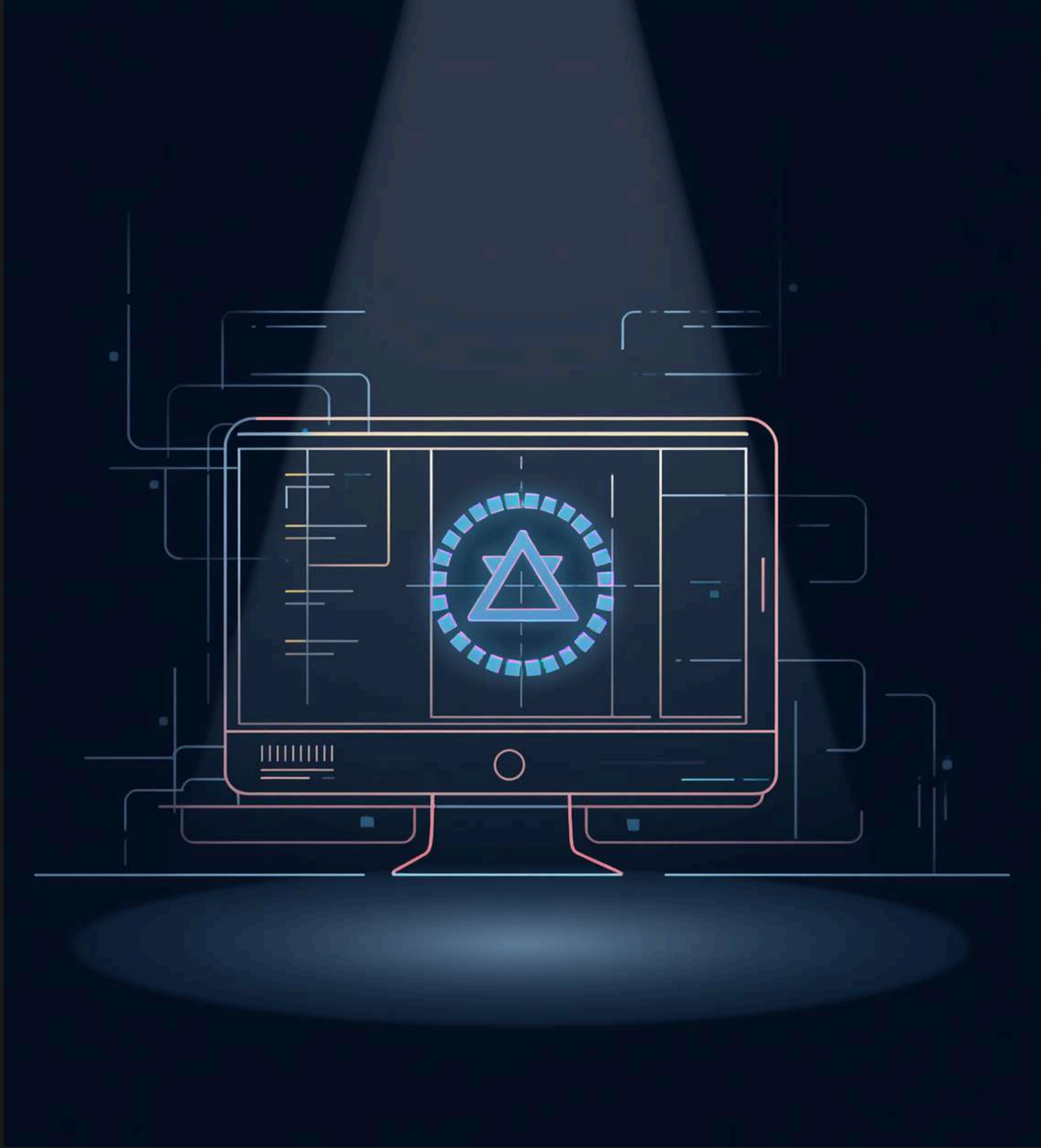
- A deadlock is a situation where processes are stuck waiting for each other
- None of the processes can proceed
- Common problem in Operating Systems & Multithreading
- Difficult to detect and resolve in real systems



75%

Systems impacted by deadlocks

75% of organizations experience significant downtime due to deadlocks, leading to resource waste and reduced productivity, which highlights the critical need for effective deadlock detection mechanisms.



Necessary Conditions for Deadlock

- Mutual Exclusion

  Only one process can use a resource at a time
- Hold and Wait

  Process holds one resource and waits for another
- No Preemption

  Resources cannot be forcibly taken
- Circular Wait

  Processes form a circular chain of waiting

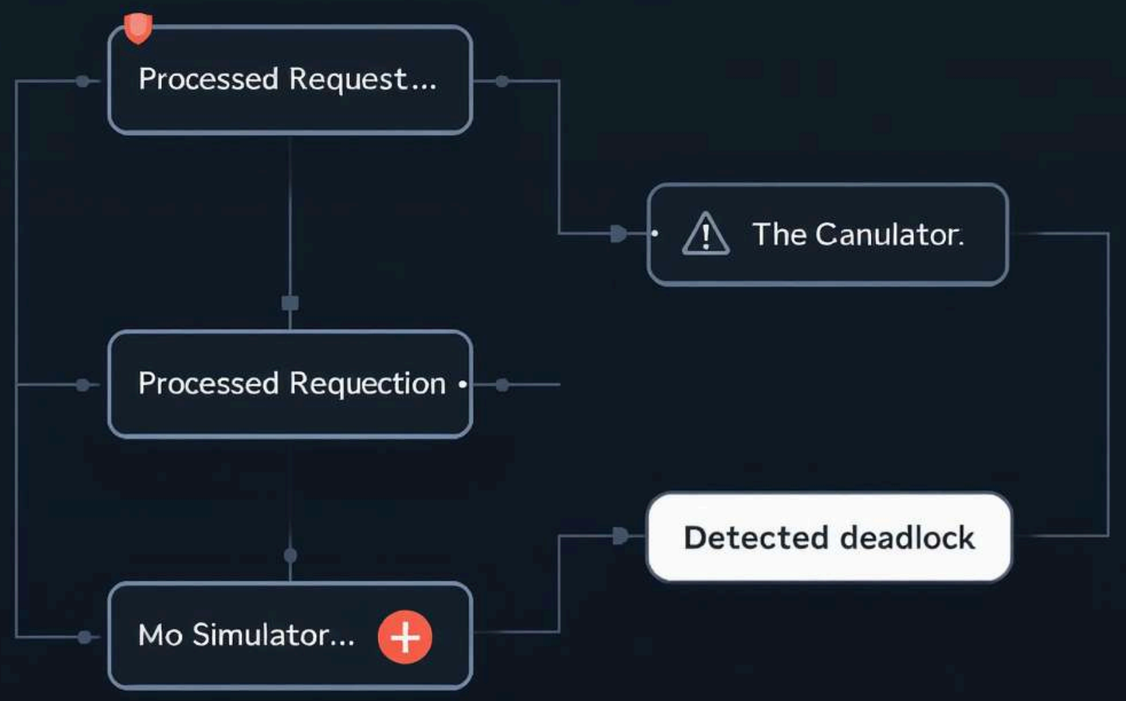
Introducing the Simulator

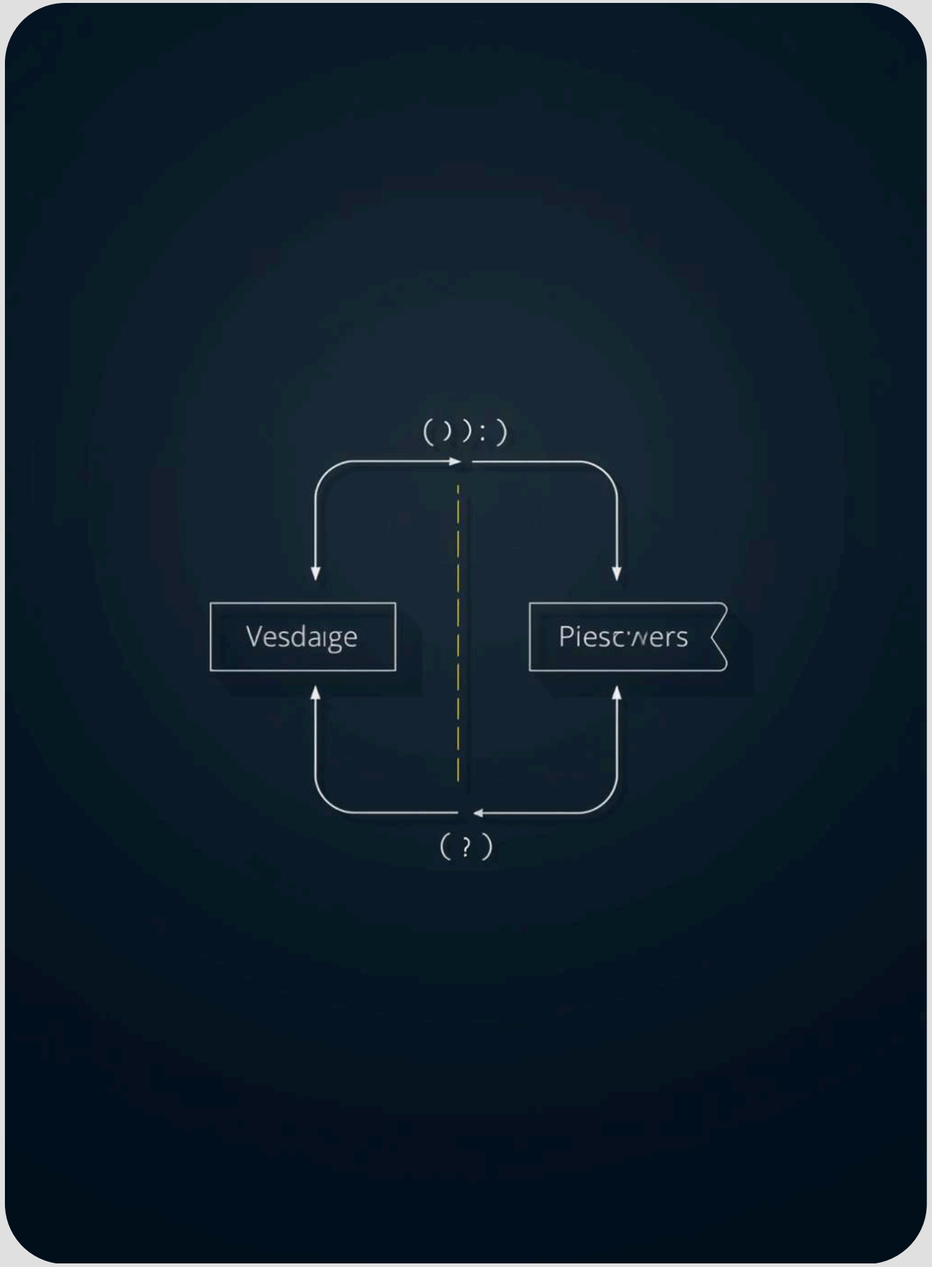
Interactive Deadlock Visualization Tool

This simulator provides users with an **interactive experience** to visualize and detect deadlocks in real-time, enhancing understanding of concurrency in systems through practical engagement and immediate feedback.

How the Simulator Functions

The simulator **efficiently tracks** resource requests and allocations, identifies cycles that suggest deadlocks, and promptly alerts users, enhancing their understanding of process interdependencies in concurrent systems.



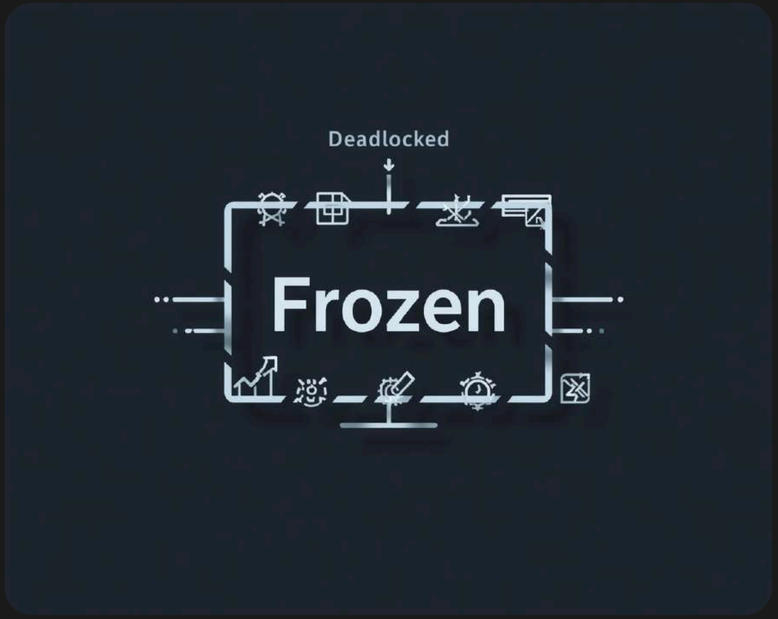


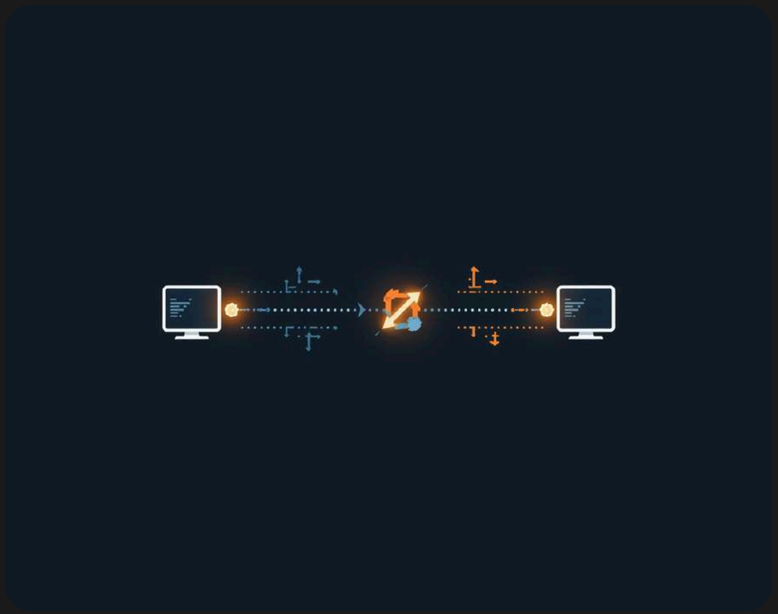
Example Scenario

This scenario illustrates **two processes** that are each waiting for resources held by the other. The visual graph highlights the deadlock situation, emphasizing the interdependencies that prevent progress.

Before and After Deadlock Detection

Visual comparison of system states





Benefits of the Simulator

Key Advantages for Learning

Visualizing Scenarios

The simulator enables students to **visualize complex deadlock scenarios** easily, enhancing comprehension of concurrent systems and providing a clearer understanding of resource allocation challenges.

Reinforcing Concepts

By interacting with the simulator, users can **reinforce theoretical concepts** with practical application, bridging the gap between theory and real-world implications of deadlock situations.

Safe Experimentation

Users can **experiment with different resource requests safely**, allowing them to explore various outcomes and gain insights into managing concurrency issues without risking system stability.

Technologies used

- Programming Language: [Java / Python / C++]
- Frontend: [HTML/CSS / GUI]
- Concepts Used:
- Operating Systems
- Graph Theory

ADVANTAGES

- Easy to understand deadlock
- Visual learning
- Helps debugging resource issues
- Useful for students and developers

LIMITATIONS

- Simulation only (not real system)
- Limited scalability
- Simplified model of OS

FUTURE IMPROVEMENTS

- Add real-time animation to visualize process execution and deadlock formation
- Improve user interface (UI/UX) for better interaction and clarity
- Support large-scale simulations with many processes and resources
- Implement advanced algorithms like Banker's Algorithm visualization
- Add step-by-step execution mode for better learning
- Provide graphical representation (Resource Allocation Graph)
- Integrate deadlock recovery techniques (process termination, resource preemption)
- Enable multi-user or collaborative simulation
- Add performance analysis metrics (waiting time, resource usage)
- Develop a web-based or mobile version for easy access
- Include AI-based suggestions to prevent deadlocks automatically
- Export simulation results as reports or logs

Thank You for Your Attention!